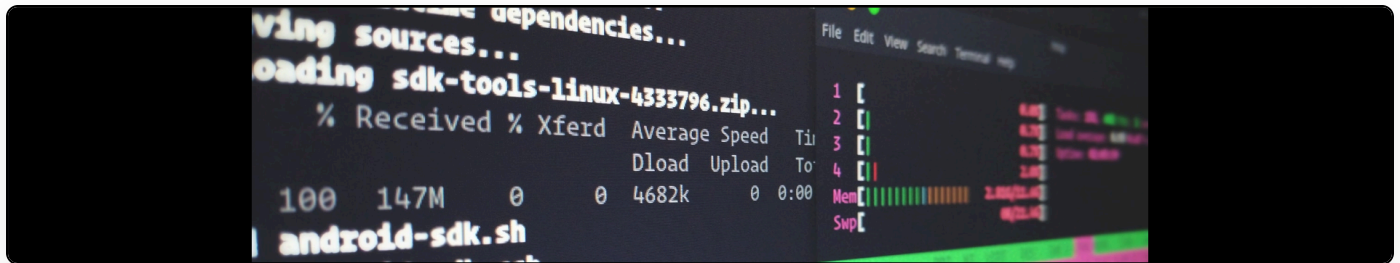


Rockaby refactoring and abstraction

Cite as: Martin Paul Eve, "Rockaby refactoring and abstraction", *eve.gd: Martin Paul Eve*, December 02, 2010, <https://doi.org/10.59348/xc4fv-rgq48>.



 Rockaby Project

I've used the snow so far this morning to start some pythonic refactoring of Rockaby.

As I mentioned in my [project announcement](#), Rockaby started life several years ago and it was a quick morning's worth of hacking about in .NET to build something that I thought nobody would ever see. As such, the code was hideous and messy. For the first code-drop, I simply transliterated the .NET code into Python/Django. This made for some grim view manipulation:

```

# build an index of where this character occurs
CharacterPages = []
Parts = []

# create a list of lists (to store page numbers)
for counter in range(0, len(pynchon.parts.Parts.FormattedPartNames)):
    Parts.append([])

for episode in episodes:
    if character_name in episode.characters:
        Parts[episode.part - 1].append(episode.episode)

    epChars = episode.characters.replace('\r\n', '\n').split('\n')
    for s in epChars:
        epCharsSplit = s.split('->')

        if epCharsSplit[0].lower() == character_name.lower():
            splitPages = epCharsSplit[1].split(',')

            for pageNum in splitPages:
                CharacterPages.append(pageNum)

ret = ''

for counter in range(0, len(pynchon.parts.Parts.FormattedPartNames)):
    part = Parts[counter]

    part.sort()

    if (len(part) > 0):
        ret = ret + '<p class="p1"><span><a href="/parts/' +
pynchon.parts.Parts.SanitizedPartNames[counter + 1] + '/' title="" +
pynchon.parts.Parts.FormattedPartNames[counter + 1] + '">' +
pynchon.parts.Parts.FormattedPartNames[counter + 1] + '</a>:'

        for i in part:
            ret = ret + ' [<a href="/episodes/' +
pynchon.parts.Parts.SanitizedPartNames[counter + 1] + '/synopsis/episode_' + str(i) +
'/" title="Synopsis of ' + pynchon.parts.Parts.FormattedPartNames[counter + 1] + '
Episode ' + str(i) + '">' + str(i) + '</a>],'

        ret = ret[0:len(ret) - 1]

    ret = ret + '<br/></span></p>'

```

Anyway, the latest commit replaces that mess with the following view:

```
episodes_used = {}  
presentation = []  
  
for charsepinfo in charsinepisodes:  
    if not charsepinfo.episode.part in episodes_used:  
        episodes_used[charsepinfo.episode.part] = []  
        episodes_used[charsepinfo.episode.part].append(charsepinfo.episode)
```

The models have now also been updated in the move towards abstraction from its original Pynchon-specific context. See the [latest commit](#) for details.